

LAPORAN UJIAN TENGAH SEMESTER COMMUNICATION PROTOCOL

Case A - REST API JSON:

pengujian endpoint GET/POST, analisis response JSON, dan parsing menjadi tabel.



DISUSUN OLEH KELOMPOK 6:

1. AZKA ARTIE RABBANI (25110500015)
2. KANAYA ANANTANI SYAFIKRI (25110500018)
3. OLGA RIZKY RAMADHANI (25110500029)
4. WANDASARI TUNGGUL HADI KUSUMO ASTUTI (25110500024)

PROGRAM STUDI SAINS DATA
FAKULTAS ILMU KOMPUTER

TAHUN 2026

1. Ringkasan Case

1.1 Latar Belakang

REST API merupakan salah satu metode komunikasi yang paling banyak digunakan dalam pengembangan aplikasi modern. REST API memungkinkan pertukaran data antara client dan server menggunakan protokol HTTP dengan format data JSON.

Pada praktikum ini kelompok memilih **Case A – REST API JSON** untuk memahami proses komunikasi client-server melalui request HTTP GET dan POST, melakukan analisis response JSON, melakukan packet capture menggunakan Wireshark, serta melakukan parsing data menggunakan Python.

1.2 Tujuan Praktikum

Tujuan praktikum ini adalah:

1. Melakukan pengujian endpoint REST API menggunakan Postman dan curl.
2. Menjalankan minimal 2 request GET dan 2 request POST.
3. Menganalisis struktur data JSON yang diterima dari server.
4. Melakukan packet capture terhadap proses komunikasi HTTP/HTTPS.
5. Mengolah response JSON menggunakan Python.
6. Mendokumentasikan seluruh evidence dalam repository GitHub.

1.3 Endpoint yang Digunakan

Contoh endpoint:

GET:

- <http://127.0.0.1:8088/api/users>
- <http://127.0.0.1:8088/api/profiles>

POST:

- <http://127.0.0.1:8088/api/users>
- <http://127.0.0.1:8088/api/profiles>

2. Pembagian Role

Role 1 – API Tester dan Postman

OLEH: AZKA ARTIE RABBANI

Tugas:

- Menyiapkan request GET dan POST.
- Menguji endpoint menggunakan Postman.
- Mendokumentasikan response dan status code.

Role 2 – Packet Capture dan Troubleshooting

OLEH: KANAYA ANANTANI SYAFIKRI

Tugas:

- Menjalankan Wireshark.
- Mengambil packet capture.
- Menganalisis paket penting.

Role 3 – Data Format Analysis dan Python Parsing

OLEH: OLGA RIZKY RAMADHANI

Tugas:

- Menganalisis struktur JSON.
- Membuat script parsing Python.
- Menghasilkan output CSV/DataFrame.

Role 4 – Report, PPT, dan Repository

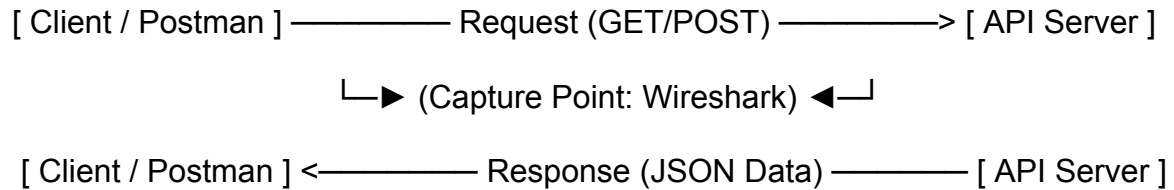
OLEH: WANDASARI TUNGGUL HADI KUSUMO ASTUTI

Tugas:

- Menyusun repository GitHub.
- Menggabungkan seluruh evidence.
- Menyusun laporan dan back up presentasi.

3. Arsitektur dan Alur Request

3.1 Arsitektur Sistem



3.2 Alur Komunikasi

1. Client mengirim request GET/POST.
2. Server menerima request.
3. Server memproses request.
4. Server mengirim response JSON.
5. Response dianalisis menggunakan Python.
6. Seluruh aktivitas di capture menggunakan Wireshark.

4. Evidence Role 1 – API Request Testing

4.1 Request GET #1

Endpoint:

<http://127.0.0.1:8088/api/users>

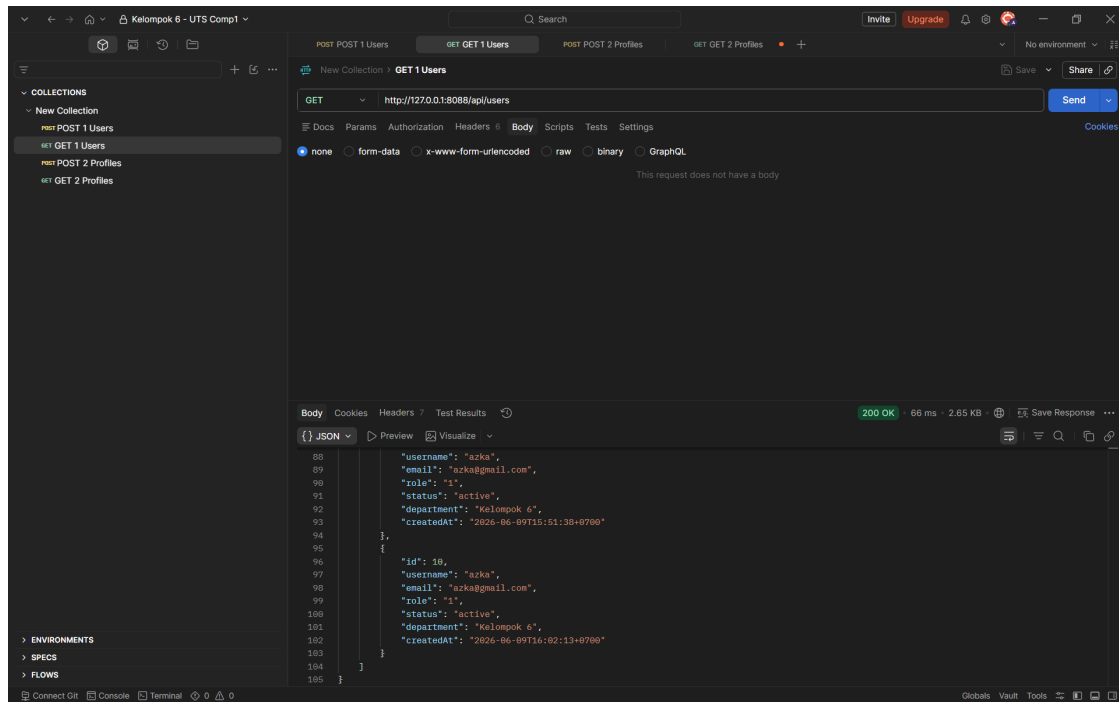
Method:

GET

Status Code:

200 OK

Hasil Response



Caption:

Gambar 1 menunjukkan response GET terhadap endpoint /posts yang berhasil mengembalikan daftar data posting dalam format JSON.

Analisis:

Request ini bertujuan untuk mengambil (membaca) daftar seluruh data users dari database lokal. Proses berhasil dilakukan dengan waktu respons 66 ms. Pada bagian body response, server mengembalikan data dalam format JSON berupa *array* yang berisi daftar pengguna.

4.2 Request GET #2

Endpoint:

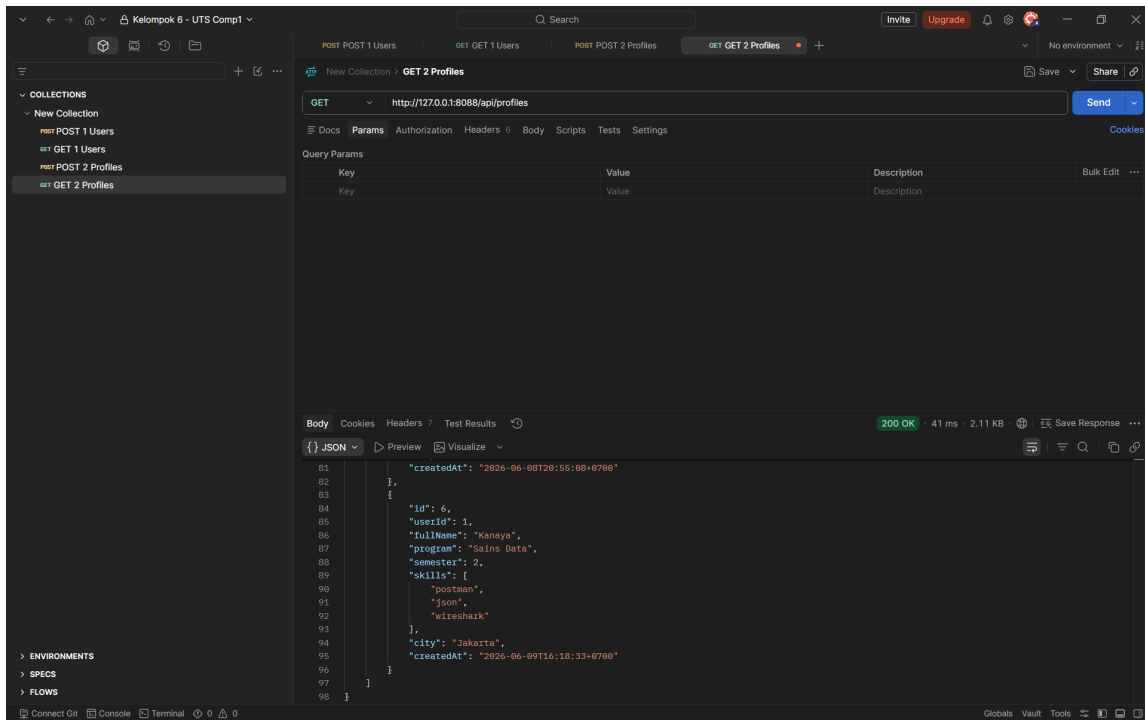
http://127.0.0.1:8088/api/profiles

Method:

GET

Status Code:

200 OK



Caption:

Gambar 2 menunjukkan data pengguna yang berhasil diterima dari server.

Analisis:

Request ini bertujuan untuk mengambil daftar data profiles. Pengambilan data berhasil dilakukan dengan waktu respons 41 ms. Server mengembalikan array JSON yang menampilkan detail profil.

4.3 Request POST #1

Endpoint:

http://127.0.0.1:8088/api/users

Method:

POST

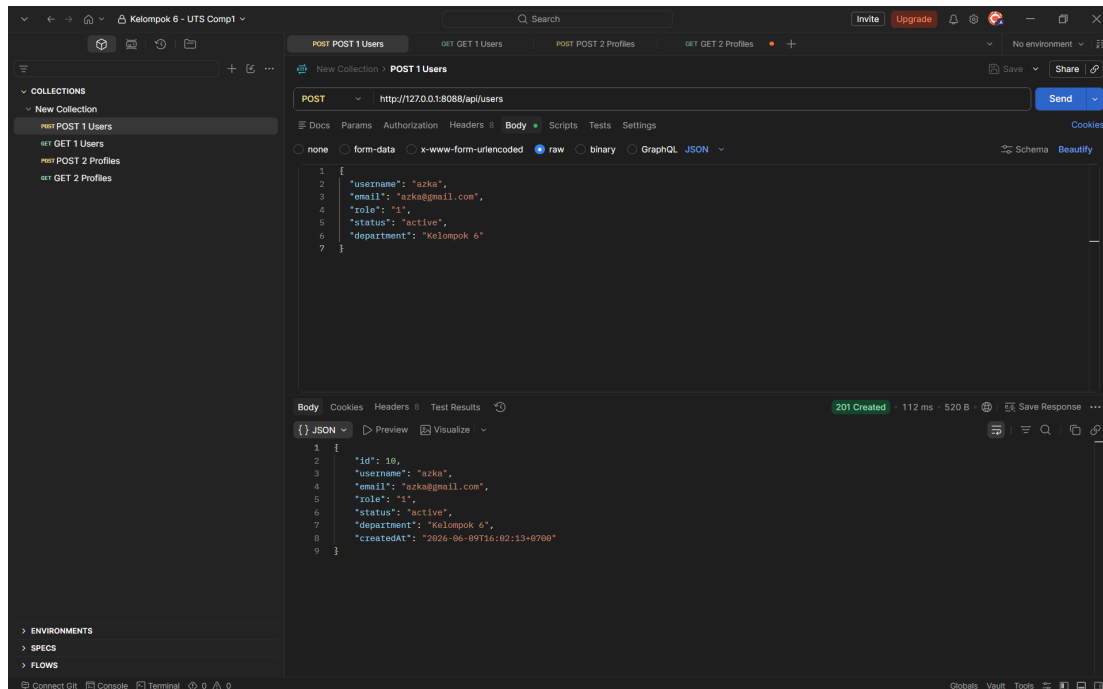
Payload:

```

{
  "username": "azka",
  "email": "azka@gmail.com",
  "role": "1",
  "status": "active",
  "department": "Kelompok 6"
}

```

Status Code:
201 Created



Caption:
Gambar 3 menunjukkan keberhasilan pembuatan resource baru menggunakan metode POST dan mengirim ke API server

Analisis: Request ini digunakan untuk mengirimkan atau membuat data user baru ke dalam sistem. Pada panel Body Request, payload JSON dikirimkan berisi "username": "azka", email, role, status, dan department. Server merespons dengan status 201 yang menandakan resource baru berhasil dibuat.

4.4 Request POST #2

Endpoint:
`http://127.0.0.1:8088/api/profiles`

Method:
POST

Payload:

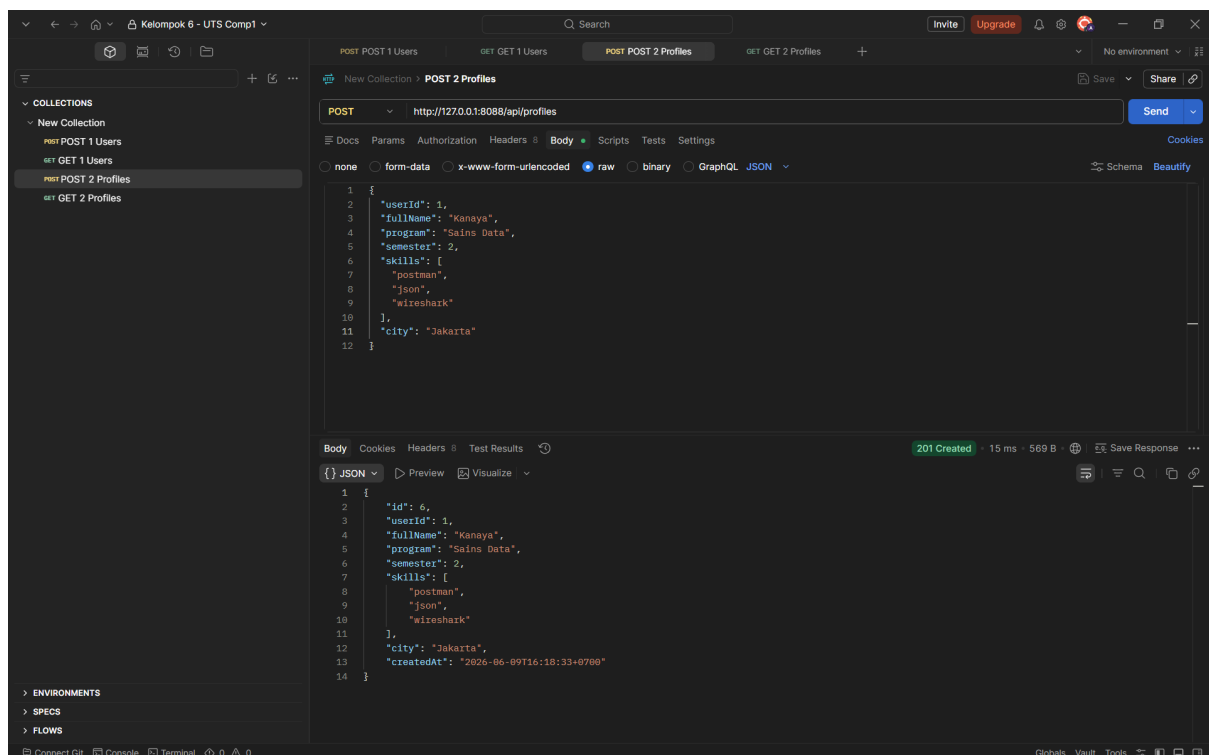
```
{  "userId": 1,  "fullName": "Student Demo",  "program": "Sains Data",
```

```

"semester": 2,
"skills": [
  "postman",
  "json",
  "wireshark"
],
"city": "Jakarta"
}

```

Status Code:
201 Created



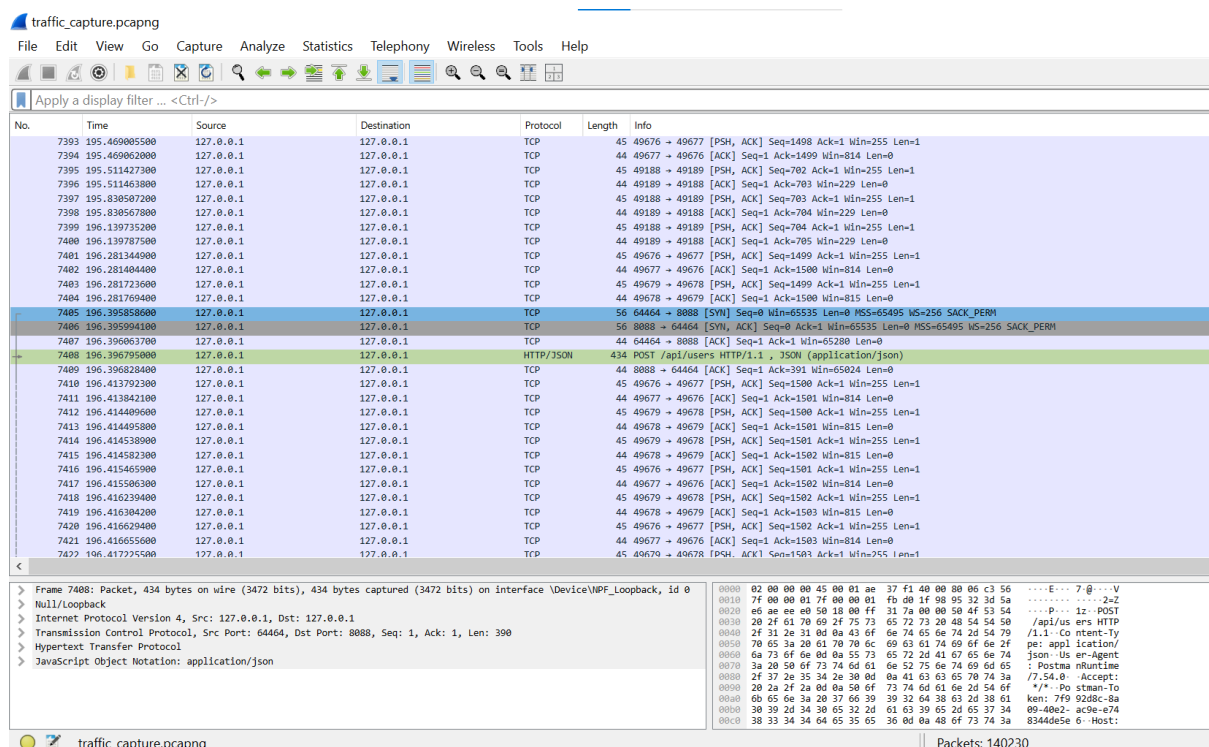
Caption:
Gambar 4 menunjukkan server berhasil membuat data user baru.

Analisis: Request ini digunakan untuk membuat data profile baru yang akan dihubungkan ke data user yang sudah ada. Pada Body Request, dikirimkan struktur JSON yang memiliki referensi "userId": 1, beserta data spesifik profil seperti nama "Kanaya" dan datanya. Proses pembuatan berhasil dengan status 201 dan waktu respons yang sangat cepat (15 ms).

5. Evidence Role 2 – Packet Capture dan Troubleshooting

5.1 Proses Packet Capture

Packet capture dilakukan menggunakan Wireshark selama proses request GET dan POST berlangsung.



Caption:
Gambar 5 menunjukkan proses request GET dan POST pada Wireshark.

5.3 Analisis Filter

Untuk meninjau jalannya protokol komunikasi tanpa terganggu oleh ribuan paket latar belakang dari sistem operasi, Role 2 menerapkan Display Filter spesifik pada Wireshark. Penerapan filter tunggal ini sangat efektif untuk mengisolasi seluruh transaksi pada Application Layer. Melalui filter ini, Wireshark secara presisi menyaring paket-paket HTTP Request yang dilakukan oleh Role 1 (API Tester) via Postman, serta HTTP Response yang dikirim balik oleh server.

No.	Time	Source	Destination	Protocol	Length	Info
7488	196.396795000	127.0.0.1	127.0.0.1	HTTP/JSON	434	POST /api/users HTTP/1.1 , JSON (application/json)
7496	196.499730000	127.0.0.1	127.0.0.1	HTTP/JSON	219	HTTP/1.0 201 Created , JSON (application/json)
7496	196.499730000	127.0.0.1	127.0.0.1	HTTP	254	GET /api/users HTTP/1.1
7496	196.499730000	127.0.0.1	127.0.0.1	HTTP/JSON	2439	HTTP/1.0 200 OK , JSON (application/json)
7496	196.499730000	127.0.0.1	127.0.0.1	HTTP	788	GET /api/profiles HTTP/1.1
7496	196.499730000	127.0.0.1	127.0.0.1	HTTP/JSON	1618	HTTP/1.0 200 OK , JSON (application/json)
7496	196.499730000	127.0.0.1	127.0.0.1	HTTP/JSON	498	POST /api/profiles HTTP/1.1 , JSON (application/json)
7496	196.499730000	127.0.0.1	127.0.0.1	HTTP/JSON	266	HTTP/1.0 201 Created , JSON (application/json)
7496	196.499730000	127.0.0.1	127.0.0.1	HTTP	257	GET /api/profiles HTTP/1.1
7496	196.499730000	127.0.0.1	127.0.0.1	HTTP/JSON	1898	HTTP/1.0 200 OK , JSON (application/json)

Caption:

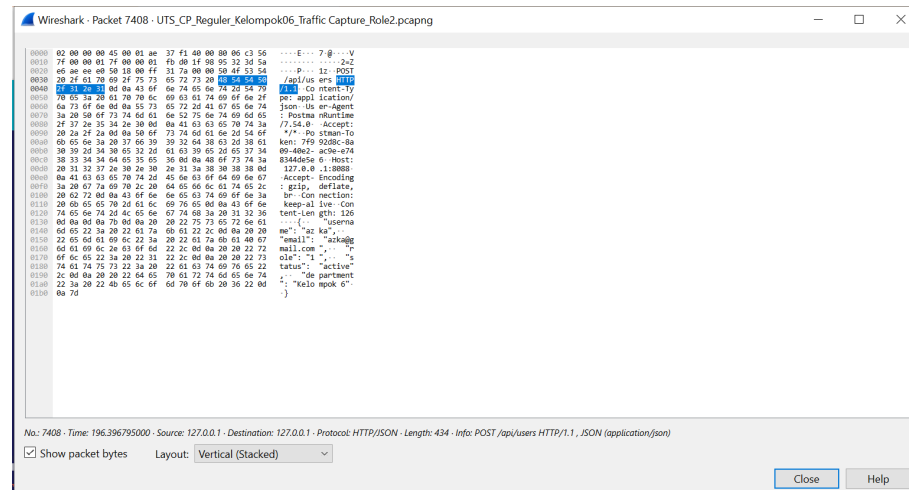
Gambar 6 menunjukkan HTTP Response yang dikirim balik oleh server.

5.3 Analisis Paket Penting

Berdasarkan gambar yang dicantumkan pada bagian 5.2, kelompok kami telah memenuhi batas minimal batas praktikum (2 GET dan 2 POST). Berikut adalah analisis paket-paket krusial tersebut:

a. Proses Request API Pertama : Kelola Pengguna (/api/users)

- Paket Request POST:

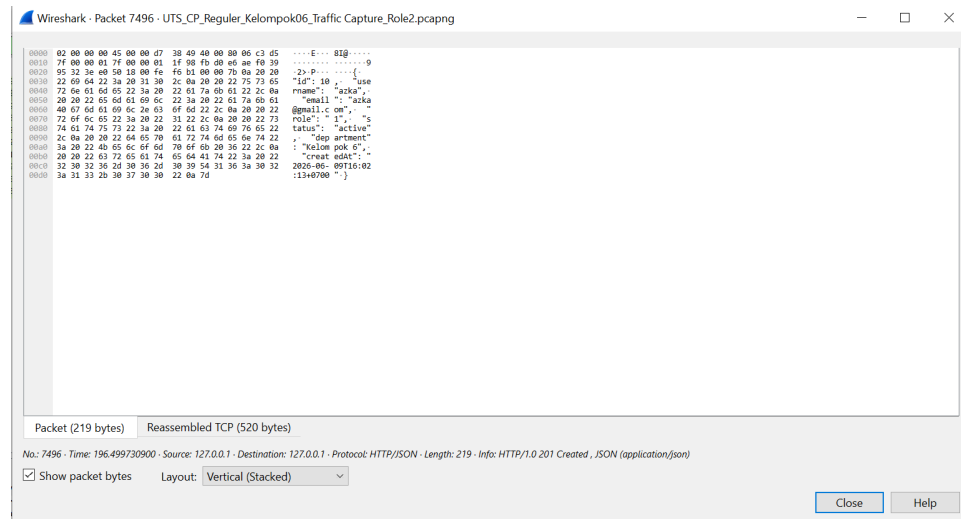


Caption:

Gambar 7 menunjukkan proses dari request post.

Terdeteksi perintah POST /api/users HTTP/1.1 dengan format data payload berupa JSON (application/json). Paket ini membuktikan proses Data Encoding saat client mencoba mengunggah entitas data baru ke server.

- Paket Response POST :

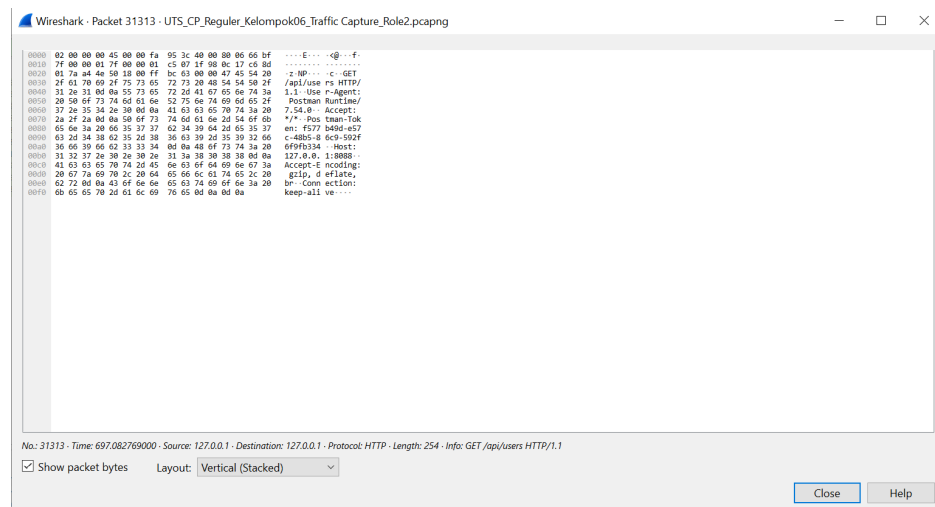


Caption:

Gambar 8 menunjukkan proses dari response post.

Server memberikan balasan resmi berupa status HTTP/1.1 201 Created. Munculnya status 201 ini memvalidasi bahwa transaksi data sukses diproses dan entitas baru berhasil disimpan di dalam server target.

- **Paket Request GET :**

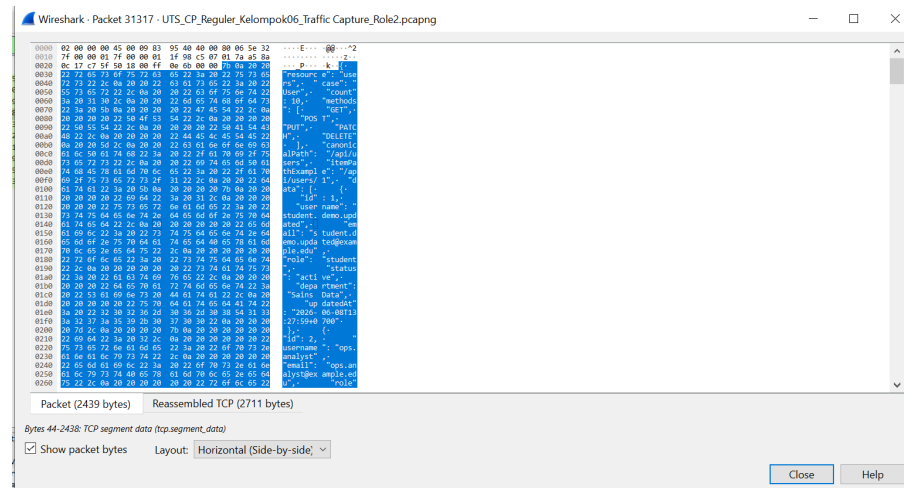


Caption:

Gambar 9 menunjukkan proses dari request get.

Terdeteksi perintah GET /api/users HTTP/1.1. Paket ini dikirimkan client untuk meminta atau menarik kembali daftar data pengguna yang ada di server.

- **Paket Response GET :**



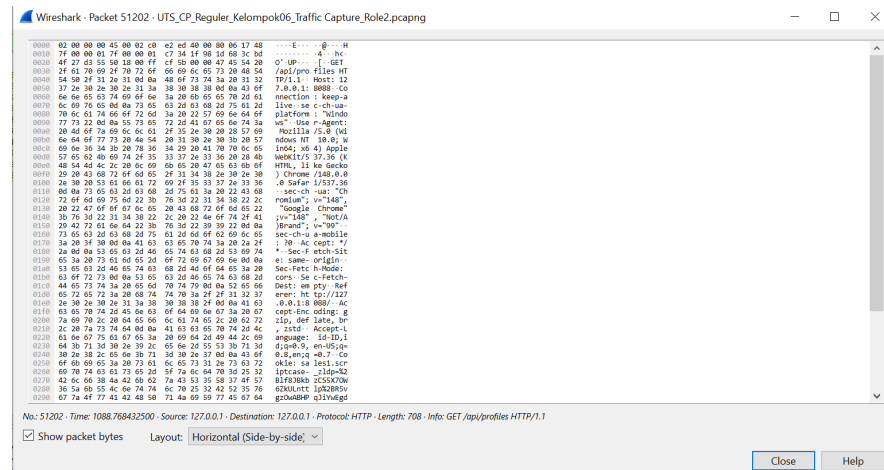
Caption:

Gambar 10 menunjukkan proses dari response get.

Server mengembalikan data dengan status HTTP/1.1 200 OK yang dibarengi dengan struktur data berformat JSON.

b. Proses Request API Kedua : Siklus Kelola Profil (/api/profiles)

- Paket Request GET :

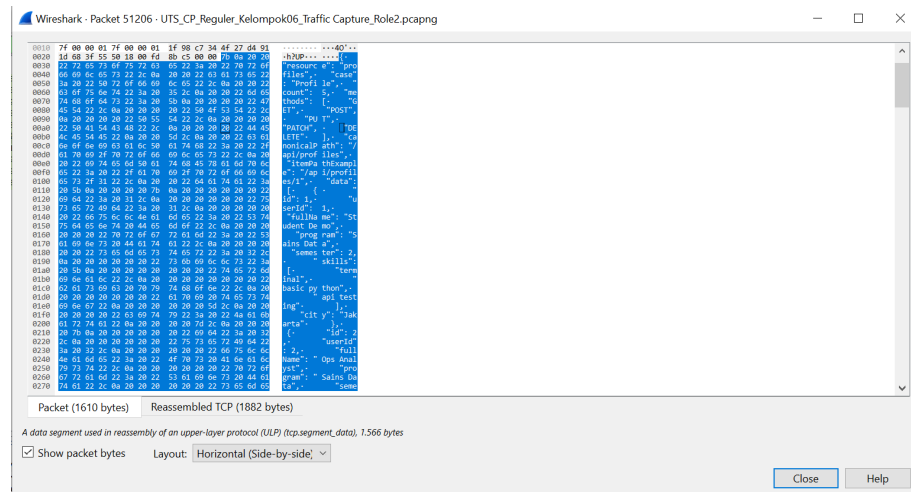


Caption:

Gambar 11 menunjukkan proses dari request get.

Terdeteksi instruksi GET /api/profiles HTTP/1.1. Request ini digunakan untuk mengambil data spesifik terkait profil dari server target.

- Paket Response GET :

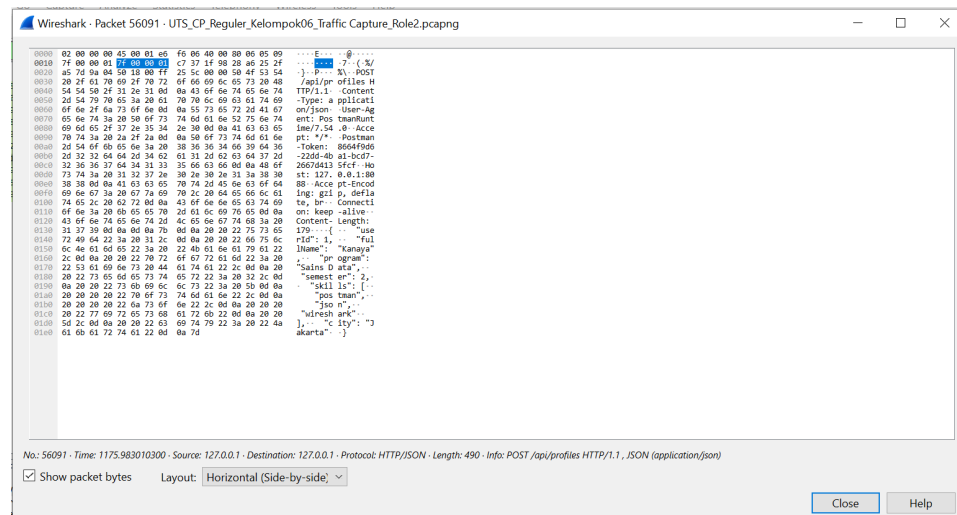


Caption:

Gambar 12 menunjukkan proses dari response get.

Server merespons balik dengan status HTTP/1.1 200 OK, membuktikan alur penarikan data profil dari database aplikasi berjalan lancar.

- Paket Request POST :



Caption:

Gambar 13 menunjukkan proses dari request post.

Terdeteksi perintah POST /api/profiles HTTP/1.1 untuk menambahkan skema atau data profil baru ke dalam sistem.

- Paket Response POST :

6. Evidence Role 3 – Data Format Analysis dan Python Parsing

6.1 Struktur JSON

A. Analisis Struktur JSON - POST 1 Users

```
{  
  "username": "azka",  
  "email": "azka@gmail.com",  
  "role": "1",  
  "status": "active",  
  "department": "Kelompok 6"  
}
```

Caption:

Gambar 15 menunjukkan struktur json dari file collection.json

Field	Tipe Data	Keterangan
username	String	Nama akun/identitas unik dari pengguna.
email	String	Alamat surat elektronik pengguna.
role	String	Kode atau level hak akses dalam sistem.
status	String	Status keaktifan akun (misalnya: active/inactive).
department	String	Nama kelompok pengguna

Analisis: Response/Payload data Users menggunakan format JSON Object datar (*flat*) yang tersusun atas pasangan *key-value* sederhana tanpa adanya struktur bersarang (*nested*).

B. Analisis Struktur JSON - POST 2 Profiles

```
{
  "userId": 1,
  "fullName": "Kanaya",
  "program": "Sains Data",
  "semester": 2,
  "skills": ["postman", "json", "wireshark"],
  "city": "Jakarta"
}
```

Caption:

Gambar 16 menunjukkan struktur json dari file collection.json

Field	Tippe Data	Keterangan
userId	Integer	ID numerik unik yang terhubung dengan data user.
full Name	String	Nama lengkap dari pengguna.
program	String	Program studi atau peminatan akademik.
semester	String	Angka semester yang sedang ditempuh.
skills	Array of Strings	Daftar keahlian teknis (berbentuk <i>nested list</i>).
city	String	Kota asal atau tempat domisili pengguna.

Analisa: Response/Payload data Profiles menggunakan format JSON Object dengan gabungan Array (pada field skills). Struktur data ini memiliki elemen bersarang (*nested text*) di mana satu *key* dapat menampung banyak nilai sekaligus.

6.2 Script Python Parsing

Berikut adalah kode program Python yang digunakan untuk membaca file `collection.json`, mengekstrak data request di dalamnya, dan melakukan transformasi menjadi format tabel terstruktur.

```
1  import json
2  import pandas as pd
3
4  # Tentukan nama file JSON yang ingin dibaca (sesuaikan jika namanya berbeda)
5  nama_file_json = 'collection.json'
6
7  # Buka dan baca isi file JSON ke dalam variabel postman_data
8  with open(nama_file_json, 'r', encoding='utf-8') as f:
9      postman_data = json.load(f)
10
11 # Struktur di bawah ini tetap sama persis dengan yang ada di gambarmu
12 for item in postman_data["item"]:
13
14     # Lewati jika item tidak memiliki body -> raw (seperti method GET)
15     if "body" not in item.get("request", {}) or "raw" not in item["request"]["body"]:
16         continue
17
18     # 1. Ambil nama item (untuk nama file & print) dan load raw string ke dict
19     name = item["name"].split()[-1].lower() # Menghasilkan 'users' atau 'profiles'
20     data_dict = json.loads(item["request"]["body"]["raw"])
21
22     # 2. Gabungkan list skills jika ada
23     if "skills" in data_dict:
24         data_dict["skills"] = ", ".join(data_dict["skills"])
25
26     # 3. Buat DataFrame, Tampilkan, dan Export ke CSV
27     df = pd.DataFrame([data_dict])
28
29     print(f"\n=== TABEL DATA {name.upper()} ===")
30     print(df)
31
32     df.to_csv(f"output_{name}.csv", index=False)
33
34 print("\nFile CSV berhasil disimpan!")
```

Caption:

Gambar 17 menunjukkan code parsing dari file `collection.json`.

6.3 Hasil Parsing

```

=== TABEL DATA USERS ===
  username      email role  status  department
0   azka  azka@gmail.com   1  active  Kelompok 6

=== TABEL DATA PROFILES ===
  userId fullName  program  semester      skills      city
0      1   Kanaya  Sains Data        2  postman, json, wireshark  Jakarta

File 'output_users.csv' dan 'output_profiles.csv' berhasil disimpan!

```

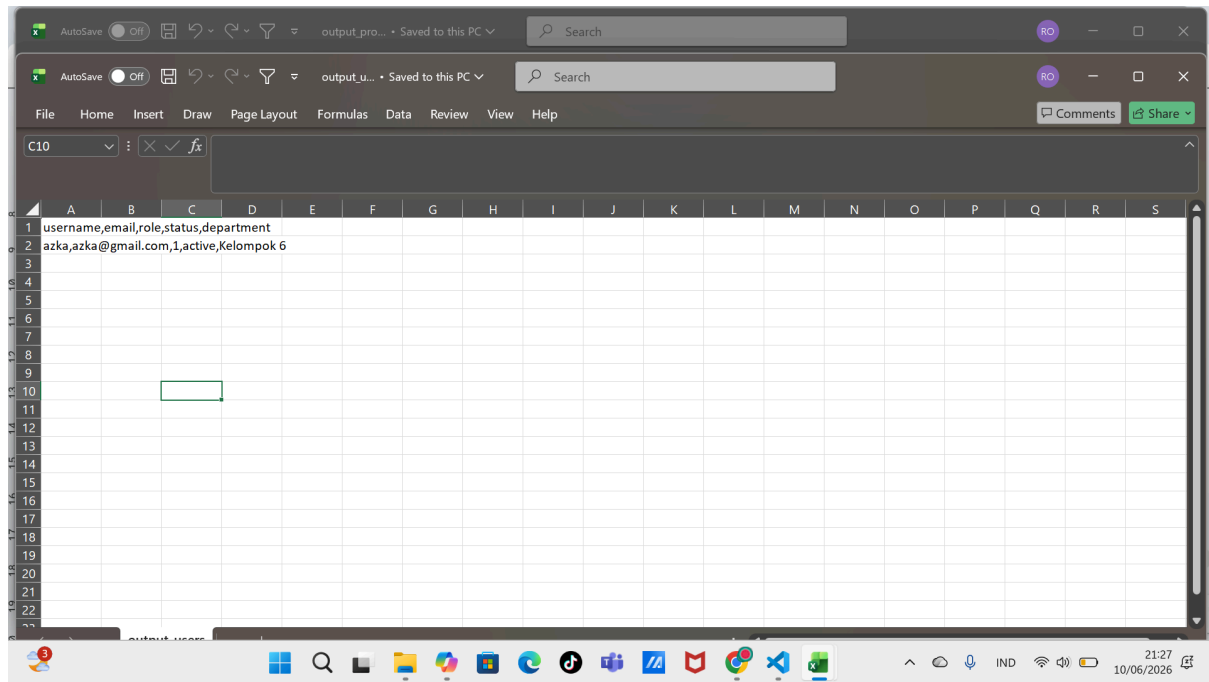
Caption:
Gambar 18 menunjukkan ouput dari code parsing.

Caption:
String JSON mentah dari objek POST 1 Users dan POST 2 Profiles pada Postman Collection telah berhasil di-parse ke dalam dua objek DataFrame Pandas yang terpisah secara terstruktur. Nilai dari kolom skills yang semula berbentuk *nested array* (bersarang) juga telah sukses dilebur menjadi satu kesatuan string yang dipisahkan oleh koma (postman, json, wireshark), sehingga dapat tersaji rapi di dalam sel tabel baris ke-0.

6.4 Output CSV

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
1	userId,fullName,program,semester,skills,city																		
2	1,Kanaya,Sains Data,2,"postman, json, wireshark",Jakarta																		
3																			
4																			
5																			
6																			
7																			
8																			
9																			
10																			
11																			
12																			
13																			
14																			
15																			
16																			
17																			
18																			
19																			
20																			
21																			
22																			

Caption:
Gambar 19 output csv pada file output_profile



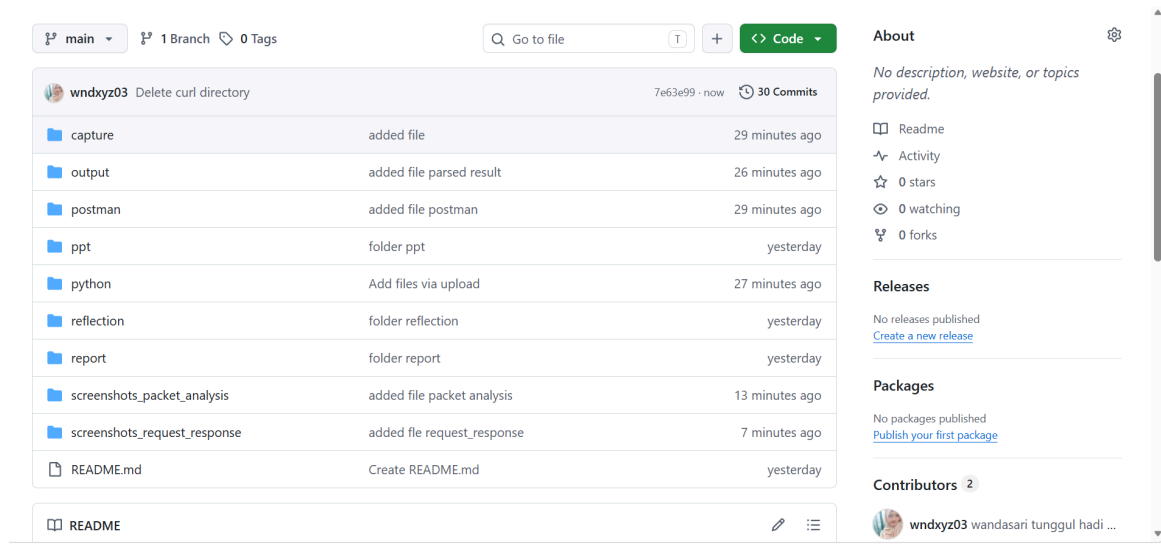
Caption:
Gambar 20 output csv pada file output_user

Analisis Output:

Berdasarkan log akhir sistem pada tangkapan layar sebelumnya, tercetak pesan “File 'output_users.csv' dan 'output_profiles.csv' berhasil disimpan!”. Hal ini membuktikan bahwa program tidak hanya berhasil melakukan pemrosesan di memori (parsing), melainkan juga sukses melakukan operasi write file ke penyimpanan lokal.

7. Evidence Role 4 – Repository dan Dokumentasi

7.1 Struktur Repository



Caption:
Gambar 21 struktur repository

Struktur folder:

UTS-Communication-Protocol-Kelompok-6

- README.md
- capture/
- curl/
- postman/
- screenshots/
- python/
- output/
- report/
- ppt/
- reflection/

7.2 Link Repository

<https://github.com/wndxyz03/UTS-Communication-Protocol-Kelompok-6>

7.3 Kompilasi Deliverables

Seluruh evidence dari setiap role telah dikumpulkan ke dalam repository sehingga dapat diverifikasi dan direplikasi oleh dosen.

8. Kendala pada saat praktikum

Beberapa kendala yang ditemui selama praktikum:

1. Terdapat ketidaksesuaian code sehingga output yang diinginkan pada proses pharsing data tidak sesuai.
2. Adanya kendala dalam menjalankan packet capture saat request dikirim oleh role 1, dikarenakan tidak bisa melakukan capture dengan dua device yang berbeda.

Solusi yang dilakukan:

1. Melakukan proses riset dan pengkodean ulang serta berdiskusi dengan ketua kelompok dan anggota kelompok yang lain agar menghasilkan output yang diinginkan.
2. Melakukan proses capture dengan satu device yang sama dengan device yang mengirim request. Setelah itu baru dilakukan proses analisis.

9. Kesimpulan

Praktikum Case A berhasil menunjukkan proses komunikasi antara client dan server menggunakan REST API berbasis HTTP dengan format JSON.

Kelompok berhasil menjalankan minimal dua request GET dan dua request POST, melakukan packet capture menggunakan Wireshark, menganalisis struktur data JSON, serta melakukan parsing data menggunakan Python menjadi format CSV dan DataFrame.

Melalui praktikum ini diperoleh pemahaman yang lebih baik mengenai protokol HTTP, struktur data JSON, proses packet capture, serta pemanfaatan Python dalam pengolahan data hasil response API.

Referensi

1. <http://127.0.0.1:8088/api/users>
2. <http://127.0.0.1:8088/api/profiles>
3. Dokumentasi Postman
4. Dokumentasi Wireshark
5. Dokumentasi Python Requests
6. Repository GitHub Kelompok